



Transaction Management

Agenda

- ▶ **What is Transaction?**
- ▶ **ACID Properties**
- ▶ **Concurrency Problems**
- ▶ **Schedules and Serializability**

Transaction

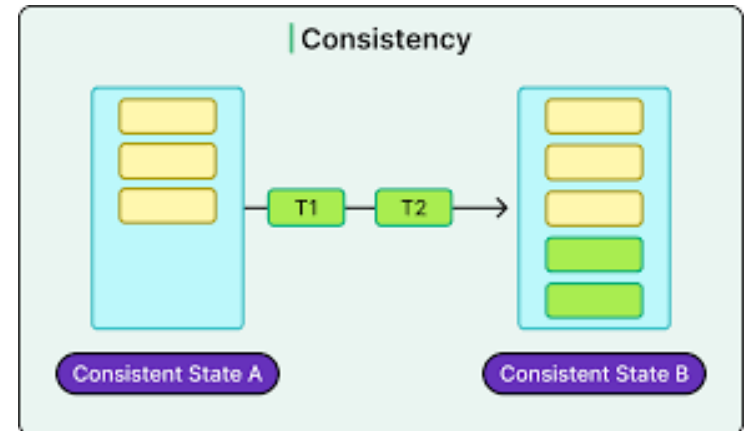
Transaction

- In DBMS, a transaction is a sequence of one or more database operations (such as INSERT, UPDATE, DELETE, or SELECT) that are executed as a single logical unit of work.



Transaction

- ▶ A transaction ensures that the database remains in a consistent state, even in the event of errors, system failures, or concurrent access.



Transaction

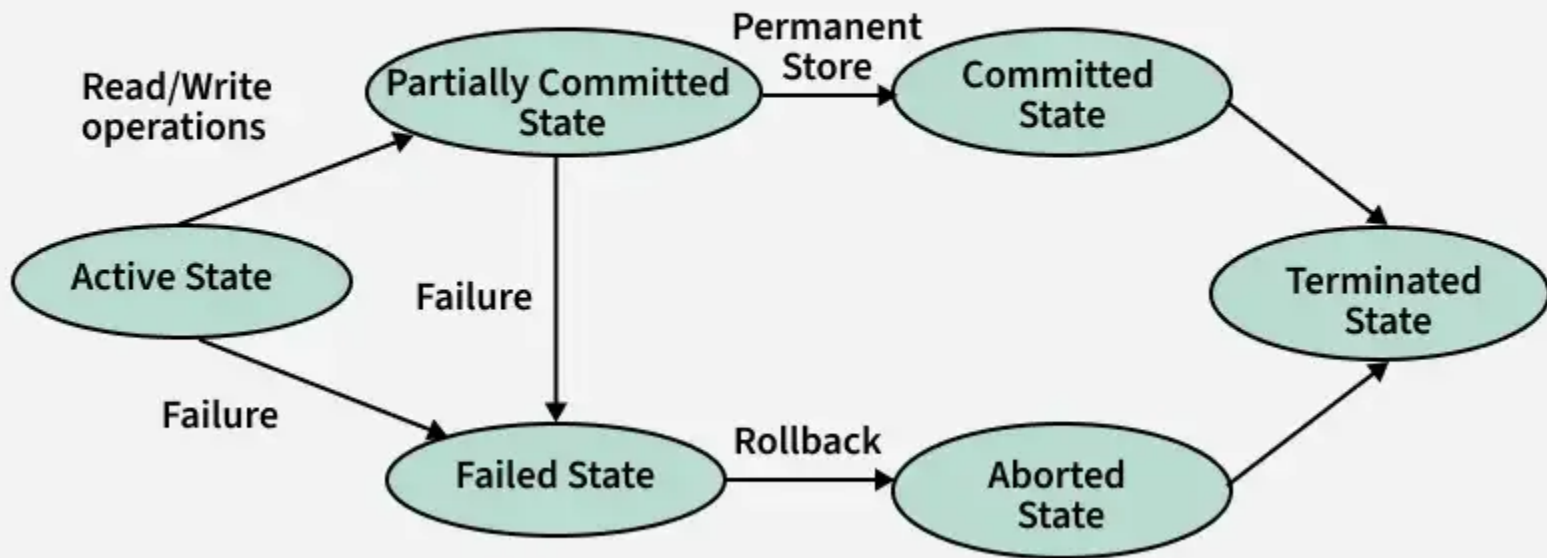
- ▶ After a failure, a DBMS ensures that the database is restored to a consistent state when the system restarts.
- ▶ A single execution of a user program may generate several transactions that perform specific database operations.
- ▶ Important characteristics of a transaction is that it consists of finite number of steps.

Transaction - States

State	Description
Active	Transaction is executing operations.
Partially Committed	All operations done, waiting for commit.
Committed	Changes made permanent in the database.
Failed	Transaction cannot proceed due to an error or failure.
Aborted	All changes undone (rolled back) and transaction terminated.

Transaction States

Transition States in DBMS



Transaction - Operations

- ▶ **Read/Access data (R):** Accessing the database item from disk (where the database stored data) to memory variable.
- ▶ **Write/Change data (W):** Write the data item from the memory variable to the disk.
- ▶ **Commit:** Commit is a transaction control language that is used to permanently save the changes done in a transaction

Transaction - Sample

BEGIN TRANSACTION;

UPDATE Accounts SET Balance = Balance - 500 WHERE AccountID = 1;
UPDATE Accounts SET Balance = Balance + 500 WHERE AccountID = 2;

COMMIT;

Transaction - Sample

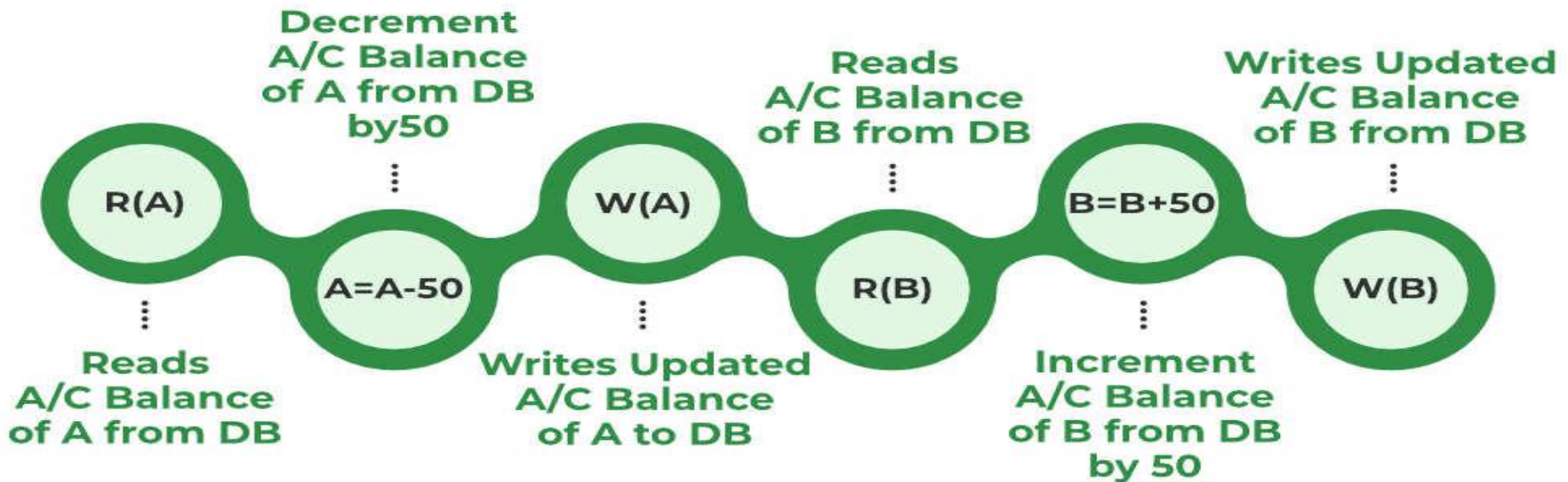
How many transactions?

UPDATE Accounts SET Balance = Balance - 500 WHERE AccountID = 1;

UPDATE Accounts SET Balance = Balance + 500 WHERE AccountID = 2;

Transaction - Example

- Transfer of 50 PKR from Account A to Account B. Initially A= 500 PKR, B= 800 PKR. This data is brought to RAM from Hard Disk.



Transaction - Example

- ▶ **Note: The updated value of Account A = 450 PKR and Account B = 850 PKR.**
- ▶ **All instructions before committing come under a partially committed state and are stored in RAM. When the commit is read the data is fully accepted and is stored on a Hard Disk.**
- ▶ **If the transaction is failed anywhere before committing we have to go back and start from the beginning. We can't continue from the same state. This is known as Roll Back.**

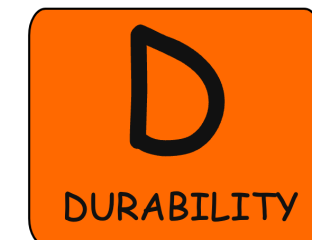
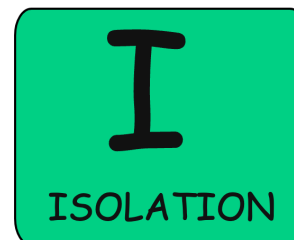
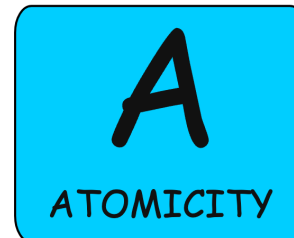


ACID Properties

ACID Properties

- ▶ A transaction in a Database Management System (DBMS) must follow four key properties known as ACID, which ensure reliable processing and maintain database integrity even in cases of errors or failures.

- ▶ A - Atomicity
- ▶ C - Consistency
- ▶ I - Isolation
- ▶ D - Durability



ACID Properties - Atomicity

- ▶ States that all operations of the transaction take place at once if not, the transactions are aborted.
- ▶ There is no midway, i.e., the transaction cannot occur partially. Each transaction is treated as one unit and either run to completion or is not executed at all.

ACID Properties - Atomicity

- ▶ **Atomicity involves the following two operations:**
 - ▶ **Abort:** If a transaction stops or fails, none of the changes it made will be saved or visible.
 - ▶ **Commit:** If a transaction completes successfully, all the changes it made will be saved and visible.
- ▶ **Recovery Management Sub-system of DBMS support atomicity by using a variety of schemes, e.g. Shadow-database Scheme.**

ACID Properties - Consistency

- ▶ **The rules (integrity constraint) that keep the database accurate and consistent are followed before and after a transaction.**
- ▶ **When a transaction is completed, it leaves the database either as it was before or in a new stable state.**

ACID Properties - Consistency

- ▶ This property means every transaction works with a reliable and consistent version of the database.
- ▶ The transaction is used to transform the database from one consistent state to another consistent state. A transaction changes the database from one consistent state to another consistent state.

ACID Properties - Isolation

- ▶ It shows that the data which is used at the time of execution of a transaction cannot be used by the second transaction until the first one is completed.
- ▶ In isolation, if the transaction T1 is being executed and using the data item X, then that data item can't be accessed by any other transaction T2 until the transaction T1 ends.
- ▶ The concurrency control subsystem of the DBMS enforced the isolation property

ACID Properties - Durability

- ▶ **Durability ensures that once a transaction has been committed, its changes are permanent, even if the system crashes or loses power immediately afterward.**
- ▶ **The recovery subsystem of the DBMS has the responsibility of Durability property.**

ACID Properties - Durability

► How it works

- DBMS uses logs (Write-Ahead Logging) and checkpoints.
- Before committing, changes are recorded in a log file on non-volatile storage (like a hard drive).
- On recovery, the DBMS replays or redoes committed changes if needed.



Concurrency Problems

INTERLEAVED CONCURRENCY

- ▶ **Concurrent systems perform multiple transactions at a time**
 - ▶ **Using multiprogramming operation system concepts**
- ▶ **The transactions are interleaved such that some commands from one process are executed, then that process is suspended, and then it executes some commands from the next process, and so on**
- ▶ **This process is called interleaved concurrency**

Transaction Samples

Figure 20.2

Two sample transactions.

(a) Transaction T_1 .

(b) Transaction T_2 .

(a)

T_1
<pre>read_item(X); X := X - N; write_item(X); read_item(Y); Y := Y + N; write_item(Y);</pre>

(b)

T_2
<pre>read_item(X); X := X + M; write_item(X);</pre>

Concurrency Problems

- ▶ **Concurrency control is an essential aspect of database management systems (DBMS) that ensures transactions can execute concurrently without interfering with each other.**
- ▶ **However, concurrency control can be challenging to implement, and without it, several problems can arise, affecting the consistency of the database**

Concurrency Problems

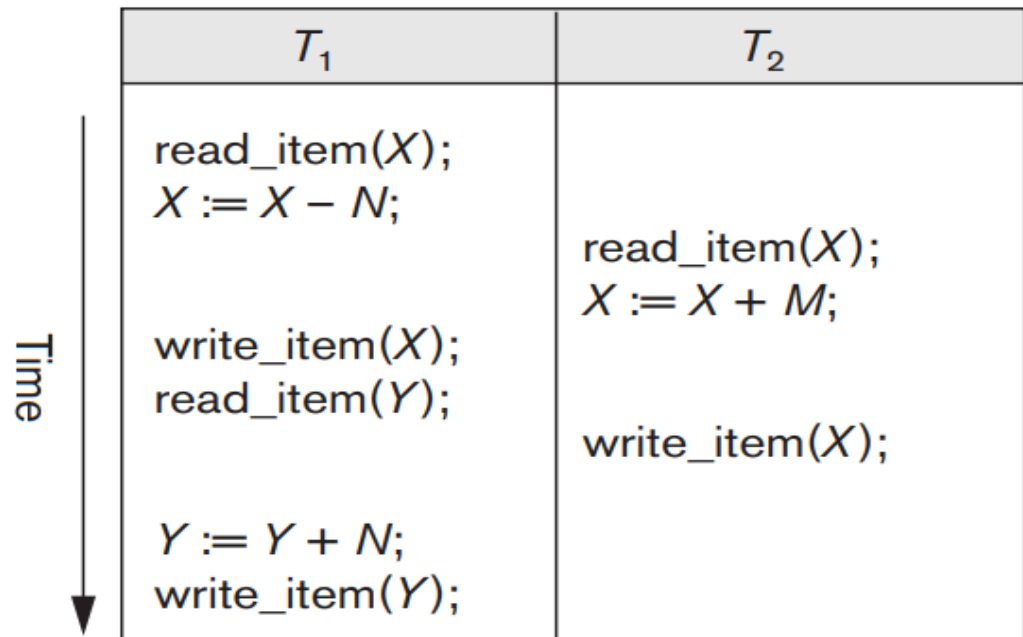
- ▶ **The Three concurrency problems that can occur in the database are:**
 - ▶ **Lost Update Problem**
 - ▶ **Temporary Update Problem**
 - ▶ **Incorrect Summary Problem**

THE LOST UPDATE PROBLEM

- ▶ **This problem occurs when two transactions that accesses the same database item, and their operations are interleaved in such a way that makes the value of some of these items incorrect.**

THE LOST UPDATE PROBLEM

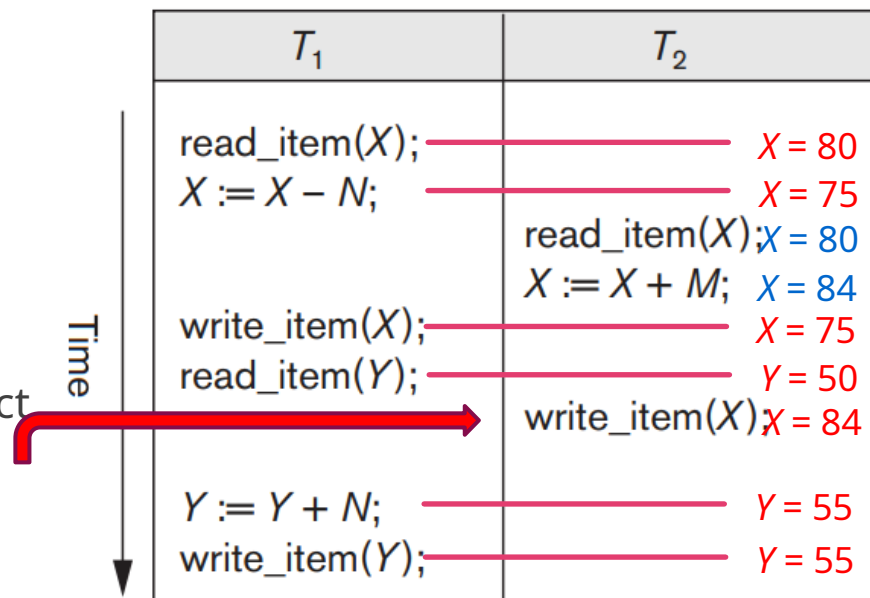
- Using the example of airline reservation system that is happening concurrently by interleaving the different operations



THE LOST UPDATE PROBLEM

Consider an airline reservation system where users concurrently update the database.

- ▶ Let $X = 80$, $Y = 50$, $N = 5$, $M = 4$.
- ▶ T_1 removes N seats from X and adds N seats to Y , while T_2 adds M seats to X .
- ▶ At the end of the operations, the correct values should be
 - ▶ $X = 80 - 5 + 4 = 79$,
 - ▶ $Y = 50 + 5 = 55$.



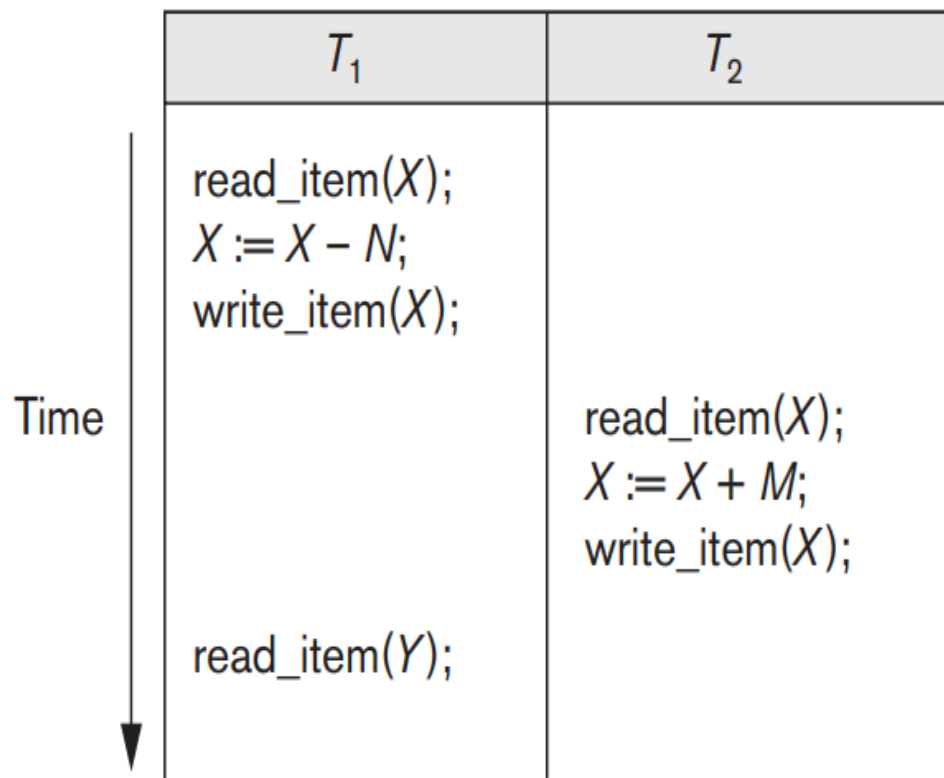
THE LOST UPDATE PROBLEM

- ▶ In the previous example, the correct value for X should have been 79 (5 seats removed, 4 seats added).
- ▶ However, the value written to the database was $X = 84$.
- ▶ The final value of X is incorrect because T2 reads the value of X before T1 changes it in the database,
- ▶ Therefore, the updated value resulting from T1 is lost.
- ▶ This is known as the lost update problem.

THE TEMPORARY UPDATE PROBLEM

- ▶ This problem occurs when one transaction updates the database but before the transaction is completed, it fails for some reason
- ▶ Meanwhile the other transaction accesses this updated item before it was reverted back to its original state
- ▶ Commonly known as the dirty read problem

THE TEMPORARY UPDATE PROBLEM



Transaction T_1 fails and must change the value of X back to its old value; meanwhile T_2 has read the *temporary* incorrect value of X .

INCORRECT SUMMARY PROBLEM

- ▶ This problem occurs when one transaction is calculating an aggregate value of the data items in the database, while another is updating one or all of these data items
- ▶ You end up with an incorrect answer even when the database is correctly updated

INCORRECT SUMMARY PROBLEM

T_1	T_3
<pre>read_item(X); X := X - N; write_item(X); read_item(Y); Y := Y + N; write_item(Y);</pre>	<pre>sum := 0; read_item(A); sum := sum + A; ⋮ read_item(X); sum := sum + X; read_item(Y); sum := sum + Y;</pre>

← T_3 reads X after N is subtracted and reads Y before N is added; a wrong summary is the result (off by N).



Schedules and Serializability

Schedule

- ▶ **Scheduling is the process of determining the order in which transactions are executed.**
- ▶ **When multiple transactions run concurrently, scheduling ensures that operations are executed in a way that prevents conflicts or overlaps between them**

TRANSACTION SCHEDULES

- ▶ When transactions are executing concurrently in an interleaved fashion, then the order of execution of operations from all the various transactions is known as a schedule (or history).
- ▶ More formally, a schedule (or history) S of n transactions $T_1, T_2, \dots, T_n$ is an ordering of the operations of the transactions.

TRANSACTION SCHEDULES

- ▶ Operations from different transactions can be interleaved in the schedule S .
- ▶ However, for each transaction T_i that participates in the schedule S , the operations of T_i in S must appear in the same order in which they occur in T_i .

SERIAL SCHEDULE

A serial schedule consists of a sequence of instructions from various transactions, where the instructions belonging to one single transaction appear together in that schedule.

T_1	T_2
$\text{read}(A)$ $A := A - 50$ $\text{write}(A)$ $\text{read}(B)$ $B := B + 50$ $\text{write}(B)$ commit	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$ $\text{read}(B)$ $B := B + \text{temp}$ $\text{write}(B)$ commit

T_1	T_2
$\text{read}(A)$ $A := A - 50$ $\text{write}(A)$ $\text{read}(B)$ $B := B + 50$ $\text{write}(B)$ commit	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$ $\text{read}(B)$ $B := B + \text{temp}$ $\text{write}(B)$ commit

CONCURRENT SCHEDULE

T_1	T_2
$\text{read}(A)$ $A := A - 50$ $\text{write}(A)$	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$
$\text{read}(B)$ $B := B + 50$ $\text{write}(B)$ commit	$\text{read}(B)$ $B := B + \text{temp}$ $\text{write}(B)$ commit

Consistent State

Figure 17.4 Schedule 3—a concurrent schedule equivalent to schedule 1.

CONCURRENT SCHEDULE

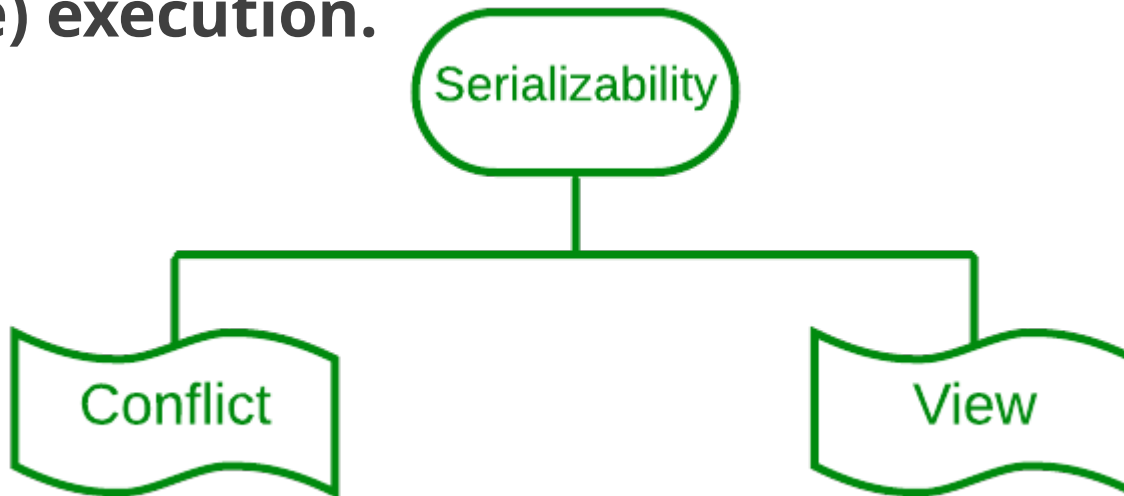
In-Consistent State

T_1	T_2
$\text{read}(A)$ $A := A - 50$	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$ $\text{read}(B)$
$\text{write}(A)$ $\text{read}(B)$ $B := B + 50$ $\text{write}(B)$ commit	$B := B + \text{temp}$ $\text{write}(B)$ commit

CS. Figure 17.5 Schedule 4—a concurrent schedule resulting in an inconsistent state.

Serializability

- **Serializability is a concept in DBMS that ensures concurrent transaction execution results in a consistent database, just like some serial (one-by-one) execution.**



SERIALIZABLE SCHEDULE

- ▶ A serial schedule is always consistent, i.e. it maintains the consistent state of the database.
- ▶ However, the same cannot be guaranteed for a concurrent schedule.
- ▶ If a concurrent schedule can be shown to have the same effect as a serial schedule, (in other words it is shown to be equivalent to a serial schedule), then it can ensure the consistency of the database.
- ▶ Such a schedule is called a *serializable schedule*.

SERIALIZABLE SCHEDULES

T_1	T_2
$\text{read}(A)$ $A := A - 50$ $\text{write}(A)$	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$
$\text{read}(B)$ $B := B + 50$ $\text{write}(B)$ commit	$\text{read}(B)$ $B := B + \text{temp}$ $\text{write}(B)$ commit

E
Q
U
I
V
A
L
E
N
T

T
O

Figure 17.4 Schedule 3—a concurrent schedule equivalent to schedule 1.

T_1	T_2
$\text{read}(A)$ $A := A - 50$ $\text{write}(A)$ $\text{read}(B)$ $B := B + 50$ $\text{write}(B)$ commit	$\text{read}(A)$ $\text{temp} := A * 0.1$ $A := A - \text{temp}$ $\text{write}(A)$ $\text{read}(B)$ $B := B + \text{temp}$ $\text{write}(B)$ commit

Figure 17.2 Schedule 1—a serial schedule in which T_1 is followed by T_2 .

SERIALIZABLE SCHEDULE

- ▶ In the previous slide, it may seem we have simply combined the operations of each transactions and executed them one after the other. However, that may not be possible each time
- ▶ This happened because there was no dependency of data from one transaction to another.
- ▶ When there are such dependencies, we can't simply merge the operations and execute them in serial manner, as this may lead to inconsistent and/or incorrect output

SERIALIZABLE SCHEDULES – EQUIVALENCE

T_1	T_2
read(A) $A := A - 50$	
	read(A) $A := A + 50$
write(A) commit	
	write(A) commit

I
S

N
O
T

E
Q
U
I
V
A
L
E
N
T

T
O

T_1	T_2
read(A) $A := A - 50$ write(A) commit	
	read(A) $A := A + 50$ write(A) commit

Conflict Serializability

- ▶ So how do we know if a schedule is serializable or not?
- ▶ Conflict serializability ensures database consistency by checking if a non-serial schedule can be rearranged into a serial order by swapping non-conflicting operations. It prevents conflicting operations (like read/write on the same data) from executing at the same time.

Conflict Serializability

- ▶ We achieve that by resolving conflicts between operations. If there are no conflicts, we can swap the operations with each other to get a resultant serial schedule
- ▶ We consider only *read(X)* and *write(X)* operations. The underlying assumption is that these two operations are the most significant operations from a scheduling perspective.
- ▶ All the other operations that happen between these significant operations will automatically be handled along with the reads and writes

CONFLICTS

Two operations are said to be conflicting if all conditions satisfy:

- **They belong to different transactions**
- **They operate on the same data item**
- **At Least one of them is a write operation**

CONFLICTS

Initial Schedule	Swapped Schedule	Conflict?												
<table><tr><th>T1</th><th>T2</th></tr><tr><td><i>read(Q)</i></td><td></td></tr><tr><td></td><td><i>read(Q)</i></td></tr></table>	T1	T2	<i>read(Q)</i>			<i>read(Q)</i>	<table><tr><th>T1</th><th>T2</th></tr><tr><td></td><td><i>read(Q)</i></td></tr><tr><td><i>read(Q)</i></td><td></td></tr></table>	T1	T2		<i>read(Q)</i>	<i>read(Q)</i>		No
T1	T2													
<i>read(Q)</i>														
	<i>read(Q)</i>													
T1	T2													
	<i>read(Q)</i>													
<i>read(Q)</i>														
<table><tr><th>T1</th><th>T2</th></tr><tr><td><i>read(Q)</i></td><td></td></tr><tr><td></td><td><i>write(Q)</i></td></tr></table>	T1	T2	<i>read(Q)</i>			<i>write(Q)</i>	<table><tr><th>T1</th><th>T2</th></tr><tr><td></td><td><i>write(Q)</i></td></tr><tr><td><i>read(Q)</i></td><td></td></tr></table>	T1	T2		<i>write(Q)</i>	<i>read(Q)</i>		Yes
T1	T2													
<i>read(Q)</i>														
	<i>write(Q)</i>													
T1	T2													
	<i>write(Q)</i>													
<i>read(Q)</i>														
<table><tr><th>T1</th><th>T2</th></tr><tr><td><i>write(Q)</i></td><td></td></tr><tr><td></td><td><i>read(Q)</i></td></tr></table>	T1	T2	<i>write(Q)</i>			<i>read(Q)</i>	<table><tr><th>T1</th><th>T2</th></tr><tr><td></td><td><i>read(Q)</i></td></tr><tr><td><i>write(Q)</i></td><td></td></tr></table>	T1	T2		<i>read(Q)</i>	<i>write(Q)</i>		Yes
T1	T2													
<i>write(Q)</i>														
	<i>read(Q)</i>													
T1	T2													
	<i>read(Q)</i>													
<i>write(Q)</i>														
<table><tr><th>T1</th><th>T2</th></tr><tr><td><i>write(Q)</i></td><td></td></tr><tr><td></td><td><i>write(Q)</i></td></tr></table>	T1	T2	<i>write(Q)</i>			<i>write(Q)</i>	<table><tr><th>T1</th><th>T2</th></tr><tr><td></td><td><i>write(Q)</i></td></tr><tr><td><i>write(Q)</i></td><td></td></tr></table>	T1	T2		<i>write(Q)</i>	<i>write(Q)</i>		Yes
T1	T2													
<i>write(Q)</i>														
	<i>write(Q)</i>													
T1	T2													
	<i>write(Q)</i>													
<i>write(Q)</i>														

SERIALIZABLE SCHEDULES – SWAPPING

- ▶ Let I and J be consecutive instructions of a schedule S .
- ▶ If I and J are instructions of different transactions and I and J do not conflict, then we can swap the order of I and J to produce a new schedule S' .
- ▶ S is equivalent to S' , since all instructions appear in the same order in both schedules except for I and J , whose order does not matter.

SWAPPING

- ▶ Consider the following schedule →
- ▶ We can swap the instructions *read(B)* and *write(A)* as they do not conflict.

T_1	T_2
read(<i>A</i>) write(<i>A</i>)	read(<i>A</i>) write(<i>A</i>)
read(<i>B</i>) write(<i>B</i>)	read(<i>B</i>) write(<i>B</i>)

SWAPPING

- ▶ The resulting schedule is shown →
- ▶ We can again swap the instructions *read(B)* and *read(A)* as they do not conflict.

T_1	T_2
read(<i>A</i>)	
write(<i>A</i>)	
read(<i>B</i>)	read(<i>A</i>)
	write(<i>A</i>)
write(<i>B</i>)	read(<i>B</i>)
	write(<i>B</i>)

SWAPPING

- ▶ The resulting schedule is shown →
- ▶ We can now swap the instructions *write(B)* and *write(A)* as they do not conflict.

T_1	T_2
read(A)	
write(A)	
read(B)	
	read(A)
	write(A)
write(B)	
	read(B)
	write(B)

SWAPPING

- ▶ The resulting schedule is shown →
- ▶ Finally, we can swap the instructions *write(B)* and *read(A)* as they do not conflict.

T_1	T_2
read(<i>A</i>)	
write(<i>A</i>)	
read(<i>B</i>)	
	read(<i>A</i>)
write(<i>B</i>)	
	write(<i>A</i>)
	read(<i>B</i>)
	write(<i>B</i>)

SWAPPING

- ▶ The resulting schedule is shown →
- ▶ Note that this schedule is a serial schedule.
- ▶ Since the original schedule has been shown to be equivalent to a serial schedule, we conclude that the original schedule maintains the consistency of the database.

T_1	T_2
read(A) write(A) read(B) write(B)	read(A) write(A) read(B) write(B)

CONFLICT EQUIVALENT SCHEDULES

- If a schedule S can be transformed into a schedule S' by a series of swaps of nonconflicting instructions, we say that S and S' are **conflict equivalent**.

T_1	T_2	E Q U I V A L E N T T O	T_1	T_2
read(A) write(A)	read(A) write(A)		read(A) write(A) read(B) write(B)	read(A) write(A) read(B) write(B)
read(B) write(B)	read(B) write(B)			



THANK YOU